Modelagem de banco de dados · Conceitos essenciais

Preparação para a sessão de modelagem do Revista Express (Django + PostgreSQL)

Objetivo e ordem de estudo

Ao final você consegue ler um requisito de negócio e extrair entidades, relacionamentos e regras; justificar cada chave, cada `NULL` e cada regra de exclusão; decidir entre tabela e JSONB; e explicar como herança viva, histórico de mudanças e perfis viram tabelas.

Ordem: Parte 1 (seções 1 a 6, cerca de 29 min) → Parte 2 (seções 7 a 9, cerca de 20 min) → prática no papel (15 min) → questões na última página (cerca de 30 min). Os tempos são estimativas de leitura.

Se a atenção cair: pare no fim da Parte 1 e volte depois. A Parte 2 é a que usamos amanhã.

Amanhã: traga as questões respondidas por escrito, mesmo as erradas. Elas mostram por onde começar.

PARTE 1 · Ferramentas

1. O que é modelar: três níveis · 4 min

Modelar um banco é decidir quais fatos o sistema precisa lembrar, como esses fatos se relacionam e quais regras o banco deve impor sozinho. O erro mais comum é começar pelas tabelas. Comece pelas perguntas: quais coisas existem, quem as cria e altera, o que muda ao longo do tempo e precisa ser lembrado, e o que nunca pode acontecer.

Para separar a conversa de negócio dos detalhes técnicos, a modelagem tradicional trabalha em três níveis, sempre nesta ordem.

Nível	Pergunta que responde	Artefato	Exemplo no Revista Express
Conceitual	O que existe e quais regras valem?	Diagrama de entidades e relacionamentos, na linguagem do negócio	Um projeto herda de um template; o filho acompanha o pai.
Lógico	Como isso vira tabelas, colunas e chaves?	Tabelas com PK, FK e restrições, sem amarrar a um SGBD	Projeto(id, slug, nome, base → Projeto)
Físico	Como fica num SGBD específico?	Tipos, índices, JSONB, restrições parciais. No Django: `models.py` e migrations	UniqueConstraint com `condition=` no PostgreSQL

Pare e pense

A regra «NULL no filho significa herdar do pai» pertence a qual nível? *Responda mentalmente antes de ler o próximo parágrafo.*

A regra de negócio («o filho acompanha o pai») é conceitual. Usar `NULL` para representar «herda» é uma decisão lógica, uma entre várias possíveis (outra seria uma tabela de sobrescritas). Escolher JSONB ou coluna é decisão física. Uma discussão de modelagem anda em círculos quando mistura os níveis; decida em ordem.

2. Entidades, relacionamentos e cardinalidade · 5 min

Entidade é uma coisa sobre a qual o sistema guarda fatos e que tem identidade própria: Projeto, Descritor, Usuário. **Atributo** é um fato sobre a entidade (nome, tópico). **Relacionamento** é a ligação entre entidades. Para decidir entre entidade e atributo, pergunte se o conceito tem vida própria: tem filhos, tem histórico, é referenciado por vários ou muda de forma independente. Se sim, é entidade.

A **cardinalidade** diz quantos registros de cada lado se ligam, com mínimo e máximo.

Tipo	Leitura	Como vira tabela	Exemplo
1:N	Um projeto tem vários cadernos; cada caderno pertence a um projeto	FK no lado N (`caderno.projeto_id`)	Projeto → Caderno
1:1	Um projeto tem no máximo um bloqueio de edição ativo	FK com UNIQUE no lado dependente	Projeto ↔ Bloqueio
N:N	Um usuário participa de vários projetos; um projeto tem vários usuários	Tabela associativa com duas FKs	Usuário ↔ Projeto via Membro

Quase todo N:N é mais que uma ligação. O Membro carrega o papel (editor, revisor, impresso), desde quando e quem convidou. Quando a ligação tem atributos, ela é uma entidade; no Django, é um modelo intermediário (`through`). A **opcionalidade** é o mínimo da cardinalidade: «todo caderno pertence a um projeto» vira FK `NOT NULL`; «um projeto pode ter uma base» vira FK que aceita `NULL`.

```
Usuario ──< Membro >── Projeto ──< Caderno ──< Ano ──< Descritor ──< LinhaEscala
                          │
                          ├── base (opcional, aponta para outro Projeto)
                          └──< Marco

──<  lê-se «um para muitos»: o lado do «<» é o lado «muitos»
```

> **Pare e pense**
>
> Os cadernos (lingua-portuguesa, matematica) são entidade ou atributo do Projeto?
> *Responda mentalmente antes de ler o próximo parágrafo.*

No Mongo atual, `cadernos` é uma lista dentro do documento (`model.ts:92-98`). Mas cada caderno tem anos e cada ano tem descritores: tem filhos, logo é entidade. Regra prática: tudo que o Mongo aninha com mais de um nível por dentro costuma ser uma entidade escondida. Você vai caçá-las na prática no papel.

3. Chaves: o que torna um registro «o mesmo» · 5 min

A **chave primária** (PK) identifica a linha: única, nunca nula, imutável. A **chave estrangeira** (FK) aponta para a PK de outra tabela, e o banco garante que o destino existe. Uma **chave natural** é um valor do negócio que identifica (o slug, o código `D01`). Uma **chave substituta** é um número sem significado gerado pelo banco (o `id`).

A regra prática é usar a substituta como PK (o Django cria o `id` sozinho) e proteger a natural com UNIQUE. Valores do negócio mudam: o slug de um projeto pode ser renomeado, e uma PK que muda obriga a reescrever todas as FKs que apontam para ela.

Há dois casos que você vai encontrar. O primeiro é a unicidade **dentro de um escopo**: `D01` existe em todo ano e caderno, e só é único dentro de um deles.

```
class Descritor(models.Model):
    projeto = models.ForeignKey(Projeto, on_delete=models.CASCADE)
    codigo = models.CharField(max_length=8)              # "D01"
    class Meta:
        constraints = [models.UniqueConstraint(
            fields=["projeto", "codigo"], name="descritor_unico_no_projeto")]
# (escopo simplificado ao projeto; no modelo real será o ano)
```

O segundo é a unicidade **só entre os não excluídos**. Hoje o Express guarda `slugOriginal` e libera o slug ao excluir (`model.ts:123`). No PostgreSQL, um índice único parcial faz o mesmo sem campo extra.

```
class Projeto(models.Model):
    slug = models.SlugField()
    excluido_em = models.DateTimeField(null=True, blank=True)
    class Meta:
        constraints = [models.UniqueConstraint(
            fields=["slug"], condition=Q(excluido_em__isnull=True),
            name="slug_unico_ativo")]
```

> **Pare e pense**
>
> Por que `D01` sozinho não pode ser a PK de Descritor? *Responda mentalmente antes de ler o próximo parágrafo.*

Porque se repete em cada ano e caderno de cada projeto. A identidade do descritor é (escopo, código). Uma pergunta útil para qualquer tabela: «o que torna dois registros o mesmo registro?». Se a resposta tem duas partes, é um UNIQUE composto.

4. Normalização e quando usar JSONB · 6 min

Normalizar é guardar cada fato uma única vez, no lugar de que ele realmente depende. A fórmula tradicional: cada atributo depende da chave, de toda a chave e de nada além da chave. As três primeiras formas normais cobrem quase tudo que você vai encontrar.

Forma	Regra	Violação típica
1FN	Cada célula guarda um valor atômico; sem colunas repetidas (padrao1, padrao2...)	`skills = "EF05LP01, EF05LP02"` como texto único
2FN	Com chave composta, todo atributo depende da chave inteira, não de parte dela	LinhaEscala(projeto, ano, descritor, padrao, texto, `ano_min`): o mínimo depende só de (projeto, ano)
3FN	Atributo não-chave não depende de outro atributo não-chave	Projeto(template_id, `template_nome`): o nome depende do template, não do projeto

Desnormalizar é duplicar dado de propósito para ganhar velocidade de leitura. É uma decisão válida, mas só depois de medir um problema real e sempre documentada, porque cada cópia pode divergir da original.

O Django mapeia `JSONField` para o tipo `jsonb` no PostgreSQL. Ele é útil, mas troca integridade por flexibilidade.

Critério	Tabela ou coluna	JSONB
Precisa filtrar pelo conteúdo	Sim, com índice normal	Possível com índice GIN, sem checagem de tipo por elemento
Precisa de FK ou CHECK sobre o conteúdo	Sim	Não
Precisa de histórico por campo	Sim, uma coluna por fato	Só do blob inteiro
Estrutura varia por registro; sempre lido e gravado inteiro	Desnecessário	Sim

Aplicado ao seu caso: `prerequisites` (lista de textos exibida inteira, nunca consultada) é candidato a JSONB. `LinhaEscala` precisa de tabela: herda por padrão, tem histórico e pode ter item anexo. `bncc.skills` (códigos como `EF05LP01`) depende de uma pergunta: alguém vai buscar descritores por habilidade? Se sim, tabela.

> **Pare e pense**
>
> Na tabela LinhaEscala(..., `ano_min`, `ano_max`), qual forma normal é violada e como corrigir? *Responda mentalmente antes de ler o próximo parágrafo.*

A 2FN: mínimo e máximo dependem só do ano, e a linha repete o valor em cada registro. A correção é mover os dois para a tabela Ano, que é como o Mongo atual já faz (`scaleRange`, `model.ts:80`).

5. Integridade, exclusão e concorrência · 6 min

O banco é a última linha de defesa. O código da aplicação tem bugs, e o próximo cliente (um script de importação, um painel de administração) não passa pela sua validação. Uma restrição (`NOT NULL`, `UNIQUE`, `CHECK`, FK) roda sempre. Do ponto de vista de segurança, validação no front-end não é controle, validação na aplicação não cobre os outros clientes do banco e restrição no banco é garantia.

Ao apagar uma linha, o que acontece com quem aponta para ela é decisão sua, e o Django exige que você a declare em cada FK.

on_delete	Efeito ao apagar o pai	Use quando
CASCADE	Apaga os filhos junto	O filho não existe sem o pai (Descritor → Ano)
PROTECT	Recusa apagar enquanto houver filhos	O pai é referência compartilhada (Template usado por projetos)
SET_NULL	O filho fica com a FK nula	A referência é opcional ou histórica

O histórico de mudanças guarda o autor por FK com SET_NULL (verificado no `django-simple-history`). Apagar um usuário apaga a autoria de tudo que ele editou. Por isso o sistema deve **desativar** usuários, nunca apagá-los; o campo `ativo` já existe hoje.

Uma **transação** agrupa operações que valem tudo ou nada; no Django, `transaction.atomic()`. A regra do «último admin» (`usuarios/service.ts:38`) precisa que a contagem de admins e a alteração não sejam intercaladas com outra alteração. No PostgreSQL, `select_for_update()` trava as linhas até o fim da transação.

Para duas pessoas editando, há duas estratégias. A **pessimista** trava antes de editar. A **otimista** deixa editar e, ao salvar, confere um número de versão; se mudou, recusa. O Mongo atual usa a otimista (`optimisticConcurrency`, `model.ts:141`) e, além dela, um bloqueio de edição.

> **Duas travas que não são a mesma coisa**
>
> A trava de linha do banco dura enquanto a transação roda: milissegundos. O «bloqueio de edição» do editor dura minutos, sobrevive a vários pedidos HTTP e expira. Isso é um dado de negócio, então vira tabela (projeto, usuário, expira_em), não lock do banco.

> **Pare e pense**
>
> Um Template usado por 12 projetos é apagado. CASCADE, PROTECT ou SET_NULL?
> *Responda mentalmente antes de ler o próximo parágrafo.*

PROTECT. CASCADE apagaria 12 revistas. SET_NULL faria as 12 perderem a base em silêncio e, com herança viva, isso muda o conteúdo que elas exibem. O banco deve obrigar a pessoa a decidir antes: mover os filhos para outro template e só então apagar. Verificado: o Django lança `ProtectedError`.

6. Do modelo ao Django · 3 min

Cada conceito acima tem uma forma direta no Django. O modelo é código no git: `makemigrations` gera o arquivo da mudança e `migrate` a aplica ao banco.

Conceito	Em Django
Entidade	`class Projeto(models.Model)` (uma tabela)
1:N	`ForeignKey(Projeto, on_delete=...)`
N:N com atributos	Modelo intermediário + `ManyToManyField(through=...)`
UNIQUE composto ou parcial	`UniqueConstraint(fields=[...], condition=Q(...))`
CHECK	`CheckConstraint(condition=...)` (parâmetro verificado no Django 6.1)
JSONB	`JSONField`
Histórico por linha	`history = HistoricalRecords()` (django-simple-history)

> **A frase que resume a Parte 1**
>
> Cada fato é guardado uma vez, no lugar de que ele depende, e toda regra que não pode ser quebrada fica no banco.

PARTE 2 · Os problemas do Revista Express

7. Herança viva (D-045) · 7 min

O problema: o projeto X herda de um template T, e T herda de um template-raiz R. Se alguém edita T, X precisa refletir a mudança. X só guarda o que sobrescreveu. A estrutura mais simples é a **lista de adjacência**: cada linha aponta para o pai (`Projeto.base`, FK para a própria tabela, nula na raiz). Para correntes curtas, de dois a quatro níveis, ela basta; closure table e caminho materializado aceleram a consulta de ancestrais ao custo de escrita mais cara.

O valor efetivo é o primeiro valor não nulo subindo a corrente. O código abaixo foi executado e passou nos três comportamentos: o filho herda quando é `None`, acompanha o pai quando o pai muda, e uma string vazia no filho conta como vazio intencional.

```
def valor(projeto, codigo, campo):
    visto = set()
    p = projeto
    while p is not None:
        if p.pk in visto:
            raise ValueError("ciclo na herança")
        visto.add(p.pk)
        d = Descritor.objects.filter(projeto=p, codigo=codigo).first()
        if d is not None and getattr(d, campo) is not None:
            return getattr(d, campo)
        p = p.base
    return None
```

`NULL` e texto vazio são coisas diferentes. `NULL` diz «não decidi, herde». O texto vazio diz «decidi deixar em branco». Misturar os dois faz um filho que apagou o texto de propósito

voltar a exibir o do pai.

> **Pare e pense**
>
> O filho sobrescreve o Padrão 02 e deixa a lista sem nenhuma linha. Isso é «herda» ou «vazio»? *Responda mentalmente antes de ler o próximo parágrafo.*

Com as linhas numa tabela filha, «sem linhas» é ambíguo: pode ser nunca sobrescrito ou sobrescrito com vazio. É preciso um marcador explícito, como uma linha em `PadraoSobrescrito(descritor, padrao)` ou uma coluna booleana por padrão. A granularidade (padrão inteiro) já está decidida no D-045; o formato do marcador é uma das decisões de amanhã.

Ciclos merecem atenção. Um `CHECK` compara só colunas da própria linha: ele barra «A herda de A» (verificado), mas não «A herda de B e B herda de A». Valide ao salvar base, subindo a corrente, e deixe o resolver com a guarda de ciclo vista acima. Para obter a corrente inteira numa só consulta, use uma CTE recursiva (`WITH RECURSIVE`), documentada no PostgreSQL e executada aqui em SQLite com a mesma forma.

```
WITH RECURSIVE ancestrais(id, base_id, nivel) AS (
    SELECT id, base_id, 0 FROM projeto WHERE id = 3
    UNION ALL
    SELECT p.id, p.base_id, a.nivel + 1
    FROM projeto p JOIN ancestrais a ON p.id = a.base_id
)
SELECT id, nivel FROM ancestrais ORDER BY nivel;
-- com y=3, x=2, tpl=1 devolveu: (3, 0), (2, 1), (1, 2)
```

> **Cuidado com SQL escrito à mão**
>
> O ORM parametriza as consultas. A injeção de SQL volta quando você monta o texto da consulta por concatenação ou f-string. Em SQL cru, o id entra sempre como parâmetro (`%s`), nunca dentro da string.

8. Histórico e auditoria · 6 min

O requisito: saber quem mudou o quê e quando, para avisar a equipe do impresso. Antes de escolher a técnica, responda quatro perguntas: o que registrar (quem, quando, campo, valor antes, valor depois, motivo), em que granularidade (por campo, por linha, por projeto inteiro), por quanto tempo e quem lê o relatório.

Padrão	Como funciona	Vantagem	Custo
Tabela espelho por modelo (django-simple-history)	Cada `save()` grava uma cópia da linha numa tabela de histórico, com `history_date`, `history_user` e `history_type` (+ criado, ~ alterado, - apagado)	Automático; `diff_against` calcula o que mudou	Uma tabela de histórico por modelo; copia a linha inteira
Log genérico de eventos	Uma tabela (modelo, id, campo, antes, depois, usuário, data)	Uma consulta lê tudo	Você escreve e mantém o código que alimenta
Snapshot do documento	Guarda a versão inteira do projeto a cada salvar	Restaurar é simples	Diff difícil; volume alto

O trecho abaixo foi executado. A coluna alterada, o valor antigo e o novo saem direto, e o tipo do registro é ~.

```
class Descritor(models.Model):
    ...
    history = HistoricalRecords()

novo, antigo = d.history.all()[0], d.history.all()[1]
for c in novo.diff_against(antigo).changes:
    print(c.field, c.old, c.new)    # topic  Tópico novo  Terceira versão
```

Para o aviso ao impresso, entra o **Marco**: uma linha `Marco(projeto, criado_em, criado_por, nota)` que registra «entreguei ao impresso neste momento». O relatório «mudou desde o marco» filtra o histórico por `history_date__gt=marco.criado_em` (verificado: devolveu só o registro posterior ao marco). Está em aberto onde guardar o status «aplicado no impresso», já que cada modelo tem a própria tabela de histórico.

> **Pare e pense**
>
> Editei o texto do template. Para o projeto X, que herda dele, algo «mudou»? *Responda mentalmente antes de ler o próximo parágrafo.*

O valor efetivo mudou, mas nenhuma linha de X mudou. Um relatório que consulte só o histórico de X perde essa alteração, e a revista impressa de X fica desatualizada, que é exatamente o que você quer evitar. O relatório de X precisa juntar o histórico próprio com o dos ancestrais, nos campos que X não sobrescreveu. A exclusão lógica (`excluido_em`) também precisa gerar registro de histórico.

9. Perfis e permissões · 6 min

Autenticação responde «quem é você»; **autorização** responde «o que você pode fazer». O Django cobre a primeira: usuário, senha com hash, sessão. A documentação recomenda definir um modelo de usuário próprio antes da primeira migration, porque trocar depois é difícil.

O modelo mais comum para a segunda é o RBAC (controle por papéis): permissões ligadas a papéis, papéis ligados a usuários. O seu sistema tem dois escopos. O **global** (admin do sistema, coordenação) vale em tudo. O **por projeto** (editor, revisor, impresso) vale só naquele projeto, e vira a tabela `Membro(projeto, usuario, papel)` com `UNIQUE(projeto, usuario)`. O papel pode ser texto com `choices` (poucos e fixos) ou uma tabela, se a equipe precisar criar papéis sem publicar versão nova. A matriz papel × ação costuma ficar melhor no código do que numa tabela de permissões.

A matriz abaixo é para você preencher antes de amanhã (✔ ou ✘). Os papéis são de exemplo; a lista real é sua.

Ação	Admin	Coordenação	Editor	Revisor	Impresso
Editar texto do projeto					
Criar projeto ou template					
Ver histórico de mudanças					
Marcar ajuste como aplicado					
Gerenciar usuários					

Ângulo de segurança: acesso ao objeto de outro

O ataque clássico em sistemas multiusuário é pedir um registro pelo id (`/projetos/17`) sem ser membro dele. A defesa é filtrar toda consulta pela tabela Membro no servidor. O front-end que esconde o botão não é controle de acesso.

Pare e pense

O admin global pode editar um projeto onde não é Membro? *Responda mentalmente antes de ler o próximo parágrafo.*

Não há resposta única; há duas regras, e o modelo precisa expressar a escolhida. Se sim, a checagem é «admin ou membro com papel de edição». Se não, o admin precisa ser Membro como todos. A segunda é mais auditável: toda edição parte de alguém com papel naquele projeto, e o histórico sempre aponta para uma pessoa com função definida.

10. Checklist para cada tabela · 2 min

Cinco perguntas para cada tabela

1. Qual é a identidade (PK) e o que mais precisa ser UNIQUE?
2. Quais colunas aceitam `NULL`, e o que o `NULL` significa em cada uma?
3. O que acontece com os filhos quando esta linha é apagada?
4. Quem escreve, quem lê, e isso muda com o tempo a ponto de precisar de histórico?
5. Algum dado aqui é cópia de outra tabela?

Cinco perguntas para cada relacionamento

1. Quantos registros de cada lado, no mínimo e no máximo?
2. A ligação tem atributos? Se tem, é uma entidade.
3. É obrigatório (`NOT NULL`) ou opcional?
4. O destino pode mudar de identidade? Se pode, não use chave natural.
5. Alguma regra depende de olhar várias linhas (ciclo, «último admin»)? Então precisa de transação, trigger ou validação.

11. O que NÃO estudar agora · 1 min

Tema	Por que fica para depois
Particularidades do MongoDB	Só vamos ler o Mongo para migrar os dados, não para modelar nele
Replicação, sharding, particionamento	O volume de uma revista por parceiro por ano não exige isso
Ajuste fino de índices e EXPLAIN	Só faz sentido com dados reais. PK e UNIQUE já criam índice, e o Django indexa FKs por padrão
Views materializadas, triggers e procedures	Aparecem só se uma regra não couber em restrição ou na aplicação
ORM avançado (select_related, prefetch_related)	Entra na hora de escrever as rotas, não na modelagem

Leitura de apoio, opcional e depois das questões:

Tema	Endereço
Modelos do Django	`docs.djangoproject.com/en/stable/topics/db/models/`
Restrições (UniqueConstraint, CheckConstraint)	`docs.djangoproject.com/en/stable/ref/models/constraints/`
Histórico por linha	`django-simple-history.readthedocs.io`
Consultas recursivas (WITH RECURSIVE)	`postgresql.org/docs/current/queries-with.html`

Prática no papel · 15 min

Abra `api/src/modules/projetos/model.ts`. O documento Mongo esconde várias entidades dentro de schemas aninhados. Desenhe o que você encontrar, em cinco passos:

Roteiro

1. Uma caixa para cada `new Schema` do arquivo (a tabela abaixo diz onde estão).
2. Ligue as caixas com setas e escreva a cardinalidade em cada uma.
3. Marque o que seria PK, FK e UNIQUE.
4. Circule o que você deixaria em JSONB.
5. Sublinhe o que muda com herança viva.

Schema (model.ts)	Linha	Pergunta para decidir
`projetoSchema`	121	O que é único? O que ele ganha com «base» e com «template»?
`cadernoSchema`	92	Tem filhos? Os ids são fixos ou criados pela equipe?
`anoSchema`	77	Quais campos dependem só do ano? (corte, passo, faixa)
`descritorSchema`	61	O que é texto herdável? O que é lista?
`linhaSchema` e `escalaSchema`	46 e 56	Como representar «este padrão foi sobrescrito»?
`itemSchema`	36	Anexo da linha: coluna ou tabela? Quantos por linha?
`bloqueioSchema`	100	Cardinalidade com Projeto? Precisa expirar?
`coresSchema`	111	Colunas, JSONB ou tabela, com herança viva?

Questões

Responda por escrito e traga amanhã. Não há gabarito aqui; a correção é na sessão. Em ordem crescente de dificuldade.

1. Um projeto tem 2 cadernos; cada caderno tem 3 anos; cada ano tem 40 descritores; cada descritor tem 4 padrões, com até 6 linhas cada. Quantas linhas, no máximo, a tabela LinhaEscala terá para um projeto que guarda tudo? E para um projeto filho que só sobrescreveu o Padrão 02 do D05 de um único ano (6 linhas)? O que essa diferença mostra sobre a herança viva?

2. A tabela LinhaEscala(projeto, ano, codigo, padrao, nivel, texto, nome_do_ano) tem chave composta por (projeto, ano, codigo, padrao, nivel), e `nome_do_ano` depende só de `ano`. Que forma normal ela viola? Como você corrige?

3. O slug `saego-2025` pode ser renomeado pela coordenação. Se o slug for a PK de Projeto e a FK de Caderno, o que acontece num renome? Qual é a alternativa e que restrição protege o slug nela?

4. No template, o campo `topic` do D01 vale «Leitura de textos». O projeto X, filho, quer a descrição vazia de propósito; o projeto Y, filho, nunca mexeu. Como a coluna representa X e Y? O que o editor deve exibir em cada um?

5. O usuário Carlos editou 200 linhas e saiu da fundação. O histórico guarda o autor por FK com `on_delete=SET_NULL`. O que acontece com a autoria dessas 200 mudanças se o usuário for apagado? O que o sistema deve fazer no lugar?

6. Dois coordenadores, ao mesmo tempo, definem `A.base = B` e `B.base = A`. Um `CHECK (base != id)` impede isso? Por quê? Onde e como você impediria? O que o resolver que sobe a corrente precisa ter?

7. Ana abre o projeto X para editar às 9h00 e deixa a aba aberta. Às 9h20 Bruno abre o mesmo projeto. Compare (a) trava de linha do banco, (b) coluna de versão (lock otimista) e (c) tabela de bloqueio com expiração. Qual resolve esse cenário e qual não resolve?

8. Maria é editora no projeto X e só leitora no Y. João é admin global. Desenhe as tabelas e escreva, em palavras, a consulta que responde «Maria pode editar Y?». Decida se João precisa ser Membro para editar X e justifique pelo histórico.

9. Regra pendente: o template mudou o D05 e o projeto X tinha sobrescrito o D05. Escreva em uma frase a regra que você escolhe, diga que colunas ou tabelas ela exige (por exemplo, guardar qual versão do pai o filho já viu) e o que o relatório do impresso mostra para X.

10. Desafio. Linha do tempo do `topic` do D05, com X herdando do template T. Dia 3: T muda de A para B. Dia 4: marco de entrega ao impresso de X. Dia 5: X sobrescreve com C. Dia 6: T muda para D. Dia 7: X remove a sobrescrita e volta a herdar. Dia 8: você gera o relatório de X «mudou desde o marco». Assuma que a sobrescrita do filho vence. Liste as linhas do relatório, com o valor efetivo para X antes e depois, e de qual tabela de histórico cada linha vem.